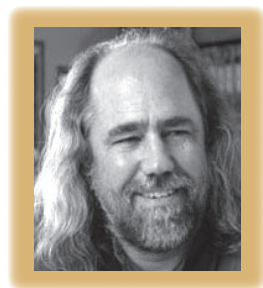


Software Abundance in the Face of Economic Scarcity, Part 2

Grady Booch



As I discussed in my previous column (Sept./Oct. 2009), we are in a period of global economic funk, by virtually every measure that we can take. The bad news, as I noted before, is that this malaise appears to be systemic and will likely linger for some time.

The good news is that we've been here before over the years—if not the centuries—and we've always seemed to get by. This is again not to discount or marginalize the human suffering that has followed in the wake of this current down market. Rather, my desire here is to celebrate the possible, and to consider the role of software in these lean times.

Although we may be in a period of material scarcity, we are graced with a cognitive surplus, an abundance that can fuel the production of software-intensive systems. Software is still the most fungible and liquid of resources, and its supply is limited only by human imagination and labor. For this reason, in my previous article, I began to make the case that software-intensive systems are a necessary element in helping us operate, innovate, and even thrive in the face of lean economic times. Here, I'll complete that discourse and offer a call to action.

Software's Limitations

I ended last time with a survey of some domains that have been touched by software's abundance, and offered examples ranging from social activity to the media. But it's not just about politics and the arts. For example, Boeing has leveraged the vast, freely accessible services available on the Web to build a mashup for supporting disaster relief (www.youtube.com/watch?v=6xB0psBjpjI&feature=channel_page&fmt=18). Similarly, by mining information about the departure and arrival of commercial flights, researchers were able to build a compel-

ling visualization of global flight patterns (www.liveleak.com/view?i=d2e_1229049414). The good folks at InSTEDD (Innovative Support to Emergencies, Disease, and Disasters) have mined the abundance of software to support the early detection of diseases (www.instedd.org).

And it's not just about the Web: the worldwide Consultative Committee of Space Data Systems (ccsds.org) has begun to formalize a rich platform for discussing the common problems in that field. Mind you, these are but four of probably many thousands of situational applications that groups have developed. In the presence of the Web's rich primordial soup of nutrients, coupled with standard protocols and mechanisms that let services play well with one another, what can be done is limited largely only by human imagination.

Costs

To be fair in my reporting, this isn't all entirely free; executing these software-intensive systems incurs real costs. For example, a simple search on Google generates approximately 0.2 g of CO₂, and an extended search has a carbon footprint of about 7 g of CO₂ (http://technology.timesonline.co.uk/tol/news/tech_and_web/article5489134.ece). Hardware costs are approximately 20 cents per gigaflop, while operating costs eat about 3 watts per gigaflop (which is why data centers chew up lots of power). Finally, although software distribution costs are low, production costs are not—people must be paid. (Open source software is a different issue, but I'll bookmark that topic for later).

Speaking of costs, my IBM colleague Clay Williams, in a personal conversation, noted that “software development is too often viewed as a cost center to be cut rather than as an asset of strategic

importance to be nourished.” The McKinsey report “Innovations in IT Management” (*McKinsey on IT*, Spring 2009) makes this point very clear: there are many opportunities in IT for fueling business, ranging from issues of daily hygiene to opportunities for creating entirely new markets. Michael Treachy and Fred Wiersman, in *The Discipline of Market Leaders* (Basic Books, 1997), suggest that effective business strategies must involve some mixture of efficiency-based, innovation-based, and customer-intimacy-based efforts. In other words, it’s not just about being the best at making the most widgets at the lowest cost; you’ll often end up delivering mediocrity, which no one will want over time. (But enough about the automotive industry ...)

Pain and Progress

Ultimately, software development is an engineering activity. That means not reaching for a perfect solution (there is no such thing, as perfection is the enemy of good enough), but rather evolving to a solution that optimally resolves the static and dynamic forces on that system. For software-intensive systems, these forces include the usual business ones (cost, schedule, and mission) but also development, environmental, operational, and legal/ethical/moral forces. Where there are tensions in resolving these forces, you will feel pain.

For software-intensive systems, customers often tell me about these points of pain:

- How do I attend to new requirements without being saddled by legacy (but also not compromising that legacy)?
- How do I integrate new technology into my existing code base?
- How do I integrate my existing systems to extract greater value from the whole?
- How do I simultaneously increase my agility in response to the market, improve efficiency and quality, and reduce costs?
- How do I attend to assets introduced through acquisition?
- How do I use software to improve market efficiency through the creation of dominant product lines?
- How do I attend to a continuously refreshed stakeholder community, a globally and temporally distributed development team, and inevitable leakage/loss of institutional memory?

- While doing all this, how do I continue to innovate?

A common thread that weaves through all these points of pain is architecture. My experience suggests that the presence of a reasonably well-understood, syndicated, governed architecture has a positive impact on each of these points of pain.

Like a River

In a previous column (“Like a River,” *IEEE Software*, May/June 2009), I used the metaphor of the enterprise as a river, with the boats traveling along the river representing the mission of that enterprise. The enterprise’s architecture, then, is the collection of engineering decisions and artifacts that steer the fleet (the organization) through the forces acting upon it and guide it toward its mission. The important subtleties in this metaphor are that

- fleets, not just single boats, are at sail;
- the forces are dynamic as well as instantaneous; and
- there are many forces beyond our control, as well as indirect forces of our own making.

In my experience, all hyper-productive organizations tend to have life cycles that involve the growth of a system’s architecture through the incremental and iterative release of testable executables. There are three important points here:

- Testable executables are the most important artifact of any development organization.
- Optimal solutions most readily emerge through the incremental and iterative delivery of stable intermediate forms.
- Architecture-as-an-artifact is an essential element in governing the course of that system and attacking risk as it evolves.

Indeed, architecture-as-an-artifact is a manifestation of technical intellectual property and thus serves as an artifact of control. It involves active yet flexible budgeting of resources, checks and balances for the coevolution of architecture and implementation, accountability for technical decisions, hedges for the future, diversification, appropriate measurements and in-

centives, cost controls, economics of scale via patterns, and actively attacking risk. I find it particularly interesting that while every robust enterprise has institutionalized best practices for its fiscal concerns, these fiscal practices have also been advanced by the consequences of governing the architecture-as-an-artifact.

Now, it’s easy to be distracted by shiny objects: all too often, I’ll see projects in crisis or projects that have just lost their way jump from one technology fad to another. However, to get to real development maturity, you have to be fiercely disciplined. In *Enterprise Architecture as Strategy* (Harvard Business Publishing, 2006), Jeanne W. Ross, Peter Weill, and David Robertson observe that you’ll see business silos in the least mature organizations, but as those organizations move up to seizing software as a strategic weapon, they’ll increasingly focus on standardized technology, the formation of an optimized core, and, finally, the emergence of an architecture of business modularity.

It’s tempting to overcontrol, however. As Clay Williams noted in a personal conversation, “We often see people pursuing efforts to make software production ‘factory-like,’ believing that will solve some problem usually related to cost and schedule control.” The “Hitachi Software Works” of the 1960s, automatic programming in the ’70s, process programming in the ’80s, and software factories in the past decade have all failed, because software development isn’t a commodity cost center but rather a center of innovation. Thus, Williams continued, “For all but routine software development and deployment, this is a really, really bad idea.” We must have freedom to draw innovation from the cognitive surplus that drives software development.

This, then, is the seminal role of architecture: it addresses how we design the solution and how we design the organization. And thus, to that end, software-intensive systems are an inescapable and necessary element in helping us operate, innovate, and even thrive in the face of lean economic times. 

Grady Booch is an IBM Fellow and one of UML’s original authors. He also developed the Booch method of software development (*Object-Oriented Analysis and Design*, Addison-Wesley, 1993). He’s working on a handbook of architectural patterns, available at www.booch.com/architecture. Contact him at architecture@booch.com.